1. **Задание**

   Вывести трехмерную сцену с движущимся объектом, который отбрасывает тень на другой объект (напр. плоскость). Тень должна перемещаться вместе с движением объекта исходя из взаимного положения источника света, объекта, который отбрасывает тень и объектов, на которые тень проецируется.

2. **Листинг программы**

```
#include <GL/glut.h>
#ifndef CALLBACK
#define CALLBACK
#endif

float xx = 0.0;
float yy = 0.0;


// Используем классы для создания матриц
class Lab
{
public:
	float x, y, z, w; // Вершинные x, y, и z координаты.
	Lab(float X, float Y, float Z, float W);
	float DotProduct4(Lab v1);
};
Lab::Lab(float X, float Y, float Z, float W)
{
	x = X;
	y = Y;
	z = Z;
	w = W;
}
float Lab::DotProduct4(Lab v1)
{
	return x * v1.x + y * v1.y + z * v1.z + w * v1.w;
}
class Matrix
{
public:
	float matrix[16];
	Matrix();
	bool CreateShadowMatrix(Lab planeNormal, Lab lightPos);
	void Clear();
};
Matrix::Matrix()
{
	Clear();
}
void Matrix::Clear()
{
	matrix[0] = 1.0f; matrix[1] = 0.0f; matrix[2] = 0.0f; matrix[3] = 0.0f;
	matrix[4] = 0.0f; matrix[5] = 1.0f; matrix[6] = 0.0f; matrix[7] = 0.0f;
	matrix[8] = 0.0f; matrix[9] = 0.0f; matrix[10] = 1.0f; matrix[11] = 0.0f;
	matrix[12] = 0.0f; matrix[13] = 0.0f; matrix[14] = 0.0f; matrix[15] = 1.0f;
}
bool Matrix::CreateShadowMatrix(Lab planeNormal, Lab lightPos)
{
	Clear();
	// Произведение нормали поверхности и света
	float dotProduct = planeNormal.DotProduct4(lightPos);
```

```
	matrix[0] = dotProduct - lightPos.x * planeNormal.x;
	matrix[4] = 0.0f - lightPos.x * planeNormal.y;
	matrix[8] = 0.0f - lightPos.x * planeNormal.z;
	matrix[12] = 0.0f - lightPos.x * planeNormal.w;
	matrix[1] = 0.0f - lightPos.y * planeNormal.x;
	matrix[5] = dotProduct - lightPos.y * planeNormal.y;
	matrix[9] = 0.0f - lightPos.y * planeNormal.z;
	matrix[13] = 0.0f - lightPos.y * planeNormal.w;
	matrix[2] = 0.0f - lightPos.z * planeNormal.x;
	matrix[6] = 0.0f - lightPos.z * planeNormal.y;
	matrix[10] = dotProduct - lightPos.z * planeNormal.z;
	matrix[14] = 0.0f - lightPos.z * planeNormal.w;
	matrix[3] = 0.0f - lightPos.w * planeNormal.x;
	matrix[7] = 0.0f - lightPos.w * planeNormal.y;
	matrix[11] = 0.0f - lightPos.w * planeNormal.z;
	matrix[15] = dotProduct - lightPos.w * planeNormal.w;
	return true;
}

// Отображение
void display(void)
{
	Matrix ShadowMatrix;

	GLfloat light_position[] = { xx, yy, 3.0, 1.0 };
	GLfloat white_light[] = { 1.0, 1.0, 1.0, 1.0 };
	glLightfv(GL_LIGHT0, GL_POSITION, light_position);
	glLightfv(GL_LIGHT0, GL_DIFFUSE, white_light);
	glLightfv(GL_LIGHT0, GL_SPECULAR, white_light);

	Lab lightPos(xx, yy, 3.0f, 1.0f);
	Lab planeNormal(0.0f, 0.0f, 1.0f, 0.0f);

	glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);


	// Задняя стенка
	glBegin(GL_POLYGON);
	GLfloat mat_specular01[] = { 1.0, 1.0, 1.0, 1.0 };
	GLfloat mat_ambient[] = { 0.9, 0.9, 0.9, 1.0 };
	GLfloat mat_diffuse[] = { 0.8, 0.8, 0.8, 1.0 };
	GLfloat mat_emission1[] = { 0.0, 0.0, 0.0, 1.0 };
	GLfloat mat_shininess[] = { 100.0 };
	glMaterialfv(GL_FRONT_AND_BACK, GL_SPECULAR, mat_specular01);
	glMaterialfv(GL_FRONT_AND_BACK, GL_AMBIENT, mat_ambient);
	glMaterialfv(GL_FRONT_AND_BACK, GL_DIFFUSE, mat_diffuse);
	glVertex3f(-1.0, -1.0, -1.0);
	glVertex3f(1.0, -1.0, -1.0);
	glVertex3f(1.0, 1.0, -1.0);
	glVertex3f(-1.0, 1.0, -1.0);
	glEnd();

	// Матрица теней
	ShadowMatrix.CreateShadowMatrix(planeNormal, lightPos);

	// Отрисовка тени и объекта
	glEnable(GL_BLEND);
	glDisable(GL_DEPTH_TEST);
	glDisable(GL_LIGHTING);

	// рисуем тень на основе матрицы
	glPushMatrix();
	glMultMatrixf(ShadowMatrix.matrix);
	glColor3f(0.0f, 0.0f, 0.0f);
	glTranslatef(0, 0, -1);
```

```
	glutSolidTeapot(0.4);
	glPopMatrix();

	glEnable(GL_LIGHTING);
	glEnable(GL_DEPTH_TEST);
	glDisable(GL_BLEND);

	// рисуем основной объект
	glPushMatrix();
	glMaterialfv(GL_FRONT_AND_BACK, GL_SPECULAR, mat_specular01);
	glMaterialfv(GL_FRONT_AND_BACK, GL_SHININESS, mat_shininess);
	glTranslatef(xx, yy, 0);
	glutSolidTeapot(0.4);
	glPopMatrix();

	glFlush();

	glutSwapBuffers();

}
// Для корректировки изменения окна
void reshape(int w, int h)
{
	glViewport(0, 0, (GLsizei)w, (GLsizei)h);
	glMatrixMode(GL_PROJECTION);
	glLoadIdentity();
	if (w <= h)
		glOrtho(-1.5, 1.5, -1.5 * (GLfloat)h / (GLfloat)w, 1.5 * (GLfloat)h /
(GLfloat)w, -10.0, 10.0);
	else
		glOrtho(-1.5 * (GLfloat)w / (GLfloat)h, 1.5 * (GLfloat)w / (GLfloat)h,
-1.5, 1.5, -10.0, 10.0);
	glMatrixMode(GL_MODELVIEW);
	glLoadIdentity();
}

void init(void)
{
	// Очищаем буфер цвета и глубины
	glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
	// Включаем свет и тестирование глубины
	glEnable(GL_LIGHTING);
	glEnable(GL_LIGHT0);
	glEnable(GL_DEPTH_TEST);
}

void specialKeys(int key, int x, int y)
{
	if (key == GLUT_KEY_RIGHT)
		xx += 0.1;
	else if (key == GLUT_KEY_LEFT)
		xx -= 0.1;
	else if (key == GLUT_KEY_UP)
		yy += 0.1;
	else if (key == GLUT_KEY_DOWN)
		yy -= 0.1;
	glutPostRedisplay();
}
int main(int argc, char** argv)
{
	glutInit(&argc, argv);
	glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB | GLUT_DEPTH);
	glutInitWindowSize(700, 500);
	glutInitWindowPosition(100, 100);
	glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
```

```
    glutCreateWindow("LR7");
    // Очищаем буфер и включаем свет
    init();
    // Функция отрисовки
    glutDisplayFunc(display);
    // Функция перемешения света
    glutSpecialFunc(specialKeys);
    // Изменение размера окна
    glutReshapeFunc(reshape);
    // Основной цикл глют
    glutMainLoop();
    return 0;
}
```

3. Пример работы программы



Рисунок 1 – Пример работы

4. Выводы

В процессе лабораторной работы был создан движущийся объект, который отбрасывает тень.